

introduction



Introduction

This is a simple test that takes about 30 minutes to complete.

We've provided a test project designed to assess how quickly and effectively you can adapt to a new project environment.

Your task is to implement the specified requirements within the project. Through this process, you'll demonstrate your ability to work fluently in a new development setup.

Therefore, the test task should be thoroughly carried out in a test project.

Please choose the tasks that best fit your role, tech stack, and confidence level.

You can find another test in the left tab of this document

Feel free to use any images, 3D models, or tools you prefer - we'll evaluate your creativity and development skills based on how you approach the tasks.

If you have any additional ideas (such as alternative tech stacks, animations, or team structures), please record your work in a video and provide either an audio explanation or a document (e.g., Google Drive link or Loom video).

Since the test includes sections for different types of candidates, there may be a few bugs in the project.

Please understand this and try to overcome it.

Frontend Developer



Technical Assessment for Frontend Developer

Task: Implement Dark/Light Mode Toggle(30 mins)

Description:

Please update the landing page by implementing the following requirements.

This is a project for the test.

[Test Project](#)

Requirements:

- Dark/Light Mode Toggle

Add a toggle switch in the navigation bar that allows users to switch between light mode and dark mode. Ensure the UI updates consistently across all components on the landing page.

- Smooth Transition

Implement a smooth visual transition when switching

between light and dark modes.

- **Responsive Design**

The updated landing page must remain fully responsive across:

Desktop, Tablet, Mobile devices

- **Code Quality**

Write clean, maintainable, and well-structured code.

Follow modern frontend best practices.

Deliverables:

- Record a video of how it works.

- Send the video link(google drive or loom.com) after completion.

Product Designer(UI/UX)



Technical Assessment for Product Designer

Task: Design a Dashboard (50 ~ 60 mins)

Description:

Take an existing dashboard, and create a fresh design.

This is a project for the test.

[Test Project](#)

First, set up a working environment for test project and make sure it's working properly.

If you aren't familiar with git and node, you can let me know for guidance.

We would like to share with you a sample video we prepared to help you with this project. We hope you find it helpful.

[Sample Video](#)

Requirements:

- Showcase your understanding of UI principles such as visual hierarchy, usability, and consistency.
- Present at least three variations of the dashboard (e.g., different styles, states, or sizes).

Deliverables:

- Record a video of how it shows.
- Send the video after completion.

Backend Engineer



Technical Assessment for Backend Engineer

Task: Create a Simple RESTful API into this backend(30mins)

Description:

Add a basic CRUD (Create, Read, Update, Delete) API for a "Notes" application into **a test project**.

This is a project for the test.

✂ All candidates are required to complete a technical assessment using the provided test project to demonstrate their ability to adapt to a new environment.

[Test Project](#)

Requirements:

- Implement endpoints:

POST /notes to create a new note.

GET /notes to retrieve all notes.

GET /notes/:id to retrieve a specific note by ID.

PUT /notes/:id to update a note.

DELETE /notes/:id to delete a note.

- Use an in-memory data structure (like an array or dictionary) to store the notes (no need for a database).
- Result shows on console or in simple page on frontend.

Deliverables:

- Record a video of how it works.
- Send the video link(google drive or loom.com) after completion.

Blockchain Developer



Technical Assessment for Blockchain Developer

Task: Contract Development & Frontend Integration
(90 ~ 120mins)

Description:

The purpose of this technical assessment is to evaluate your Solidity (Rust) development skills, smart contract security practices, and ability to integrate blockchain functionality into a frontend application. Candidates are expected to demonstrate adaptability and technical problem-solving within a new project environment.

This is a project for the test.

※ All candidates are required to complete a technical assessment using the provided test project to demonstrate their ability to adapt to a new environment.

Task Requirements:

1. Smart Contract Development

Develop a simple, secure, and efficient investment smart contract in Solidity (Rust) with the following functionality:

- Allow users to invest ETH (SOL)
- Record and manage investor information
- Track total invested funds
- Simulate distribution of investment returns

2. Deployment & Interaction Scripts

Create deployment and interaction scripts compatible with either:

- **web3.js**
- [ethers.js](#)
- **Anchor**
- **Solana CLI**
- **@solana/web3.js**

Scripts should support contract deployment and execution of core contract functions.

3. Frontend Integration

Integrate the smart contract into the provided frontend application to ensure:

- Proper wallet connection
- Smooth and intuitive user interactions
- Reliable execution of contract functions

Clear user experience and transaction flow

Deliverables:

Please submit the following:

1. **Deployed Smart Contract Details**
 - i. Contract ABI
 - ii. Deployed contract address
2. **Project Demonstration Video**
 - i. Record a short walkthrough demonstrating the working application and smart contract integration
 - ii. Briefly explain your implementation and show the project running successfully
 - iii. Upload the recording using **Loom**: <https://loom.com>
 - iv. Share the **live Loom video link**

AI Engineer



Technical Assessment for AI Engineer

Task: Build and integrate a small chatbot API into the demo project that can understand user queries and return relevant answers using mock data.(30~40mins)

This is a project for the test.

✂ All candidates are required to complete a technical assessment using the provided test project to demonstrate their ability to adapt to a new environment.

[Test Project](#)

Requirements:

1. Create a REST API

- Endpoint: POST /chat

- Input

```
{ "message": "Show me apartments under $1200 in New York" }
```

- Output

```
{ "reply": "Found 2 apartments under $1200 in New York.", "results": [...] }
```

2. Use simple NLP or an LLM (e.g., OpenAI, LangChain)

- Extract:

Intent (e.g., search_property)

Entities (e.g., city, price, bedrooms)

3. Mock Data

Create a small JSON file (e.g., properties.json) with 5–10 sample listings:

```
[  
  
  { "id": 1, "city": "New York", "bedrooms": 2, "price": 1100 },  
  
  { "id": 2, "city": "San Francisco", "bedrooms": 1, "price": 2000 }  
  
]
```

4. Logic

- Parse user query → extract filters (city, price, etc.)
- Search mock data → return matching results

- If no match → return a friendly message (“No results found”)

Deliverables:

- Record a video of how it works.
- Send the video link(google drive or loom.com) after completion.

QA Engineer

Technical Assessment for QA Engineer

Task: Assess the candidate's ability to set up the project, identify critical issues, and provide a structured test plan (50 ~ 60 mins).

Project Setup & Execution:

Clone the repository: [Test Project](#)

Run the project locally and confirm the setup works.

If there are any issues during setup, document them and provide solutions.

Manual Testing & Bug Identification:

Explore key features of the platform (e.g., user registration, wallet connection).

Identify at least 5 critical issues (UI/UX, functionality, performance, security).

Log issues with clear steps to reproduce, expected vs. actual results, and severity level.

Test Plan & Case Writing:

Create 5 test cases covering different scenarios (functional, security, performance).

Format: Test Case ID, Scenario, Steps, Expected Result, Actual Result, Status (Pass/Fail).

Suggest any edge cases that should be tested.

(Optional) Automation Testing:

If experienced in automation testing, write a basic Selenium/WebdriverIO or Postman API test to validate a feature.

Provide the test script and execution steps.

Submission:

Share a bug report document (Google Docs or Notion).

Upload test cases in Excel or Google Sheets.

(If applicable) Submit an automation script via GitHub or a ZIP file.

Record a short video (3-5 minutes) explaining findings and improvements.

